

OK!
Experimento
adotado ao
reigado.

PROJETO 4: MINIX

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Objetivos

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
- Processos órfãos e zumbis.

1.1. TAREFA 5 (gerenciamento de processos): Processos órfãos

Um processo órfão é um processo cujo processo pai terminou ou foi terminado.

Processos órfãos desperdiçam recursos e podem potencialmente complicar a operação de um servidor. Entretanto, existem algumas soluções para o problema de processos órfãos:

1. **Exterminar o processo** é a técnica mais comumente usada. Neste caso o processo órfão é terminado;
2. **Expiração** é uma técnica na qual cada processo é alocado por um certo período de tempo para que possa terminar antes que o processo seja finalizado. Caso necessário, um processo pode pedir mais tempo para terminar antes que seu tempo expire.

Ao se tornar órfão em um sistema Unix-like, um processo é automaticamente adotado pelo processo “init”. Esta operação é chamada re-parenting (em uma tradução livre, apadrinhamento) e ocorre por padrão. Embora o processo tenha o processo init como seu pai, ele ainda é chamado processo órfão pois o processo que o gerou originalmente não existe mais.

Sendo assim, para essa tarefa, iremos simular a questão de processos órfãos, usando o comando kill no processo pai da Tarefa 4, item (b) desta lista. Além disso, iremos demonstrar o re-parenting dos processos filhos e descreveremos o método usado para realizar esse experimento.

1.1. Experimento – MINIX – Usando ps

Para essa simulação, usaremos as técnicas de "Exterminar o processo" e "Expiração".

Primeiramente, aplicamos "Exterminar o processo", encerrando o processo pai manualmente. Como resultado, todos os seus processos filhos se tornam órfãos. O sistema operacional, então, adota esses processos órfãos e os coloca sob a responsabilidade do processo init (PID 1). O init é o primeiro processo iniciado pelo kernel do sistema operacional e é responsável por gerenciar a criação e finalização de outros processos.

Em seguida, os processos filhos continuam rodando até atingirem o tempo de 20 segundos definido no código. Após esse período, eles encerram naturalmente, o que caracteriza a técnica de "Expiração", pois os processos tiveram um tempo limite para execução antes de serem finalizados automaticamente.

Dessa forma, a simulação demonstra ambas as abordagens na prática, permitindo observar o comportamento dos processos órfãos e sua finalização programada.

Antes de iniciarmos o teste, iremos tentar instalar o pstree no MINIX.

No entanto, após várias horas de estudo e tentativas, não conseguimos implementá-lo, pois não há documentação sobre sua disponibilidade via `pkgin` ou `pkgin_cd` (Figura 1). Além disso, ao explorar o repositório disponível em <https://www.minix3.org/distfiles-backup/> (Figura 2), identificamos algumas possibilidades de uso, mas nenhuma solução viável para a instalação direta.

```
gustavo [ ~/pstree-2.33 ]$ pkgin_cd install pstree
Checking for CD in /dev/c0d2p2.
/dev/c0d2p2 is read-only mounted on /mnt
Found.
cleaning database from http://www.minix3.org/pkgsrc/packages/3.1.8/i686/All entries...
downloading pkg_summary.bz2: 100%
processing remote summary (/mnt/install/packages)...
updating database: 100%
pstree is not available on the repository
nothing to do.
gustavo [ ~/pstree-2.33 ]$ pkgin_cd install psmisc-20.1nb2
Checking for CD in /dev/c0d2p2.
/dev/c0d2p2 is read-only mounted on /mnt
Found.
cleaning database from http://www.minix3.org/pkgsrc/packages/3.1.8/i686/All entries...
downloading pkg_summary.bz2: 100%
processing remote summary (/mnt/install/packages)...
updating database: 100%
psmisc-20.1nb2 is not available on the repository
nothing to do.
gustavo [ ~/pstree-2.33 ]$ pkgin install psmisc-20.1nb2
psmisc-20.1nb2 is not available on the repository
nothing to do.
gustavo [ ~/pstree-2.33 ]$ pkgin install pstree
pstree is not available on the repository
nothing to do.
gustavo [ ~/pstree-2.33 ]$ pkgin install pstree-2.36
pstree-2.36 is not available on the repository
nothing to do.
gustavo [ ~/pstree-2.33 ]$
```

Figura 1 – pstree não encontrado para baixar nos repositórios em geral.

pstreams-0.7.0.tar.gz	2013-May-27 11:30:44	54.2K	application/x-gtar-compressed
pstree-2.33.tar.gz	2013-May-27 11:30:44	9.9K	application/x-gtar-compressed
pstree-2.36.tar.gz	2013-May-27 11:32:41	10.9K	application/x-gtar-compressed
psutils-0.17.tar.gz	2013-May-27 11:30:44	60.5K	application/x-gtar-compressed

Figura 2 – Backups do código pstree.c.

Mas ao tentarmos compilar em nosso MINIX, percebemos alguns erros relacionados a biblioteca `sys/ioctl.h`, que por algum motivo nessa versão do MINIX e com essa do `pstree` que instalei, surge um erro relacionado a uma falta de definição da estrutura `winsize` (Figura 3). A estrutura `winsize` é usada para obter o tamanho da janela do terminal, e é definida no cabeçalho `sys/ioctl.h`. E após algumas outras pesquisas, aparentemente comentar essa linha de `import` poderia resolver pois assim passaria a utilizar o `stdio.h`, e de fato conseguimos compilar com alguns outros erros, e ao jogarmos para a pasta `/usr/bin` o nosso arquivo binário compilado e executarmos `pstree` no console do MINIX, não obtivemos com êxito a instalação conforme na Figura 4.

```
gustavo [ ~/pstree-2.33 ]$ gcc -o pstree pstree.c
pstree.c: In function 'PrintTree':
pstree.c:691: warning: incompatible implicit declaration of built-in function 'snprintf'
pstree.c: In function 'main':
pstree.c:866: error: storage size of 'winsize' isn't known
pstree.c:867: error: invalid application of 'sizeof' to incomplete type 'struct winsize'
gustavo [ ~/pstree-2.33 ]$
```

Figura 3 – Erro 1 ao compilar o pstree.c.

```
gustavo [ ~ ]$ ls
PROJET04.4-Correcao  PROJETO4.4-src  ps-etc-main  pstree-2.33  pstree-2.36
gustavo [ ~ ]$ cd pstree-2.33
gustavo [ ~/pstree-2.33 ]$ ls
README  pstree  pstree.c
gustavo [ ~/pstree-2.33 ]$ gcc -o pstree pstree.c
pstree.c: In function 'PrintTree':
pstree.c:690: warning: incompatible implicit declaration of built-in function 'snprintf'
gustavo [ ~/pstree-2.33 ]$ which pstree
/usr/bin/pstree
gustavo [ ~/pstree-2.33 ]$ pstree
pstree: No processes read.
gustavo [ ~/pstree-2.33 ]$
```

Figura 4 – Erro 2 ao compilar o pstree.c e executar.

Como a instalação do pstree resultou em erro e ele foi apenas sugerido como uma dica, optamos por utilizar a ferramenta ps, que já vem integrada ao MINIX. Embora o ps não exiba os processos em formato de árvore, podemos acrescentar argumentos como "lf" para melhorar a visualização das informações.

Para demonstrar o re-parenting dos processos filhos, inserimos um delay de 20 segundos no tempo de vida dos processos no Código 1. Isso garante tempo suficiente para a execução de comandos em paralelo, conforme ilustrado no passo a passo a seguir.

O comando ps lf será utilizado antes da execução do código para visualizar a hierarquia dos processos em execução no sistema. Esse comando exibe informações detalhadas, incluindo o PPID (Process ID do pai), PID (Process ID do processo), estado do processo e nível de prioridade, permitindo analisar a organização dos processos antes da criação dos novos processos filhos.

Na Figura 5, apresentamos a saída do comando "ps lf" antes da execução do código. Essa referência servirá para comparação com o estado do sistema após a criação dos novos processos.

```
root [ ~ ]# ps lf
```

F	S	UID	PID	PPID	PGRP	SZ	RECV	TTY	TIME	CMD
10	S	0	305	303	305	0	(wait)	pm	p1	0:00 bash
10	W	0	335	305	305	0		pm	p1	0:00 ps
10	S	0	143	1	143	0	(tty)	vfs	co	0:00 getty
10	S	0	144	1	144	0	(tty)	vfs	c1	0:00 getty
10	S	0	145	1	145	0	(tty)	vfs	c2	0:00 getty
10	S	0	146	1	146	0	(tty)	vfs	c3	0:00 getty
10	S	0	238	236	238	0	(tty)	vfs	p0	0:00 bash

Figura 5 – Utilização do comando "lf" para mais detalhes.

Agora vamos executar o Código 1 compilado com "make fork-ex-10pid" no lado esquerdo, e iremos deixar no lado direito um outro terminal para acompanhar o uso do comando ps conforme podemos verificar na Figura 6 abaixo.

```

root [ ~/tarefa_5 ]* ./fork-ex-10pid
Processo filho criado! PID: 323 || Pai: 322
Processo filho criado! PID: 324 || Pai: 322
Processo filho criado! PID: 325 || Pai: 322
Processo filho criado! PID: 326 || Pai: 322
Processo filho criado! PID: 327 || Pai: 322
Processo filho criado! PID: 328 || Pai: 322
Processo filho criado! PID: 329 || Pai: 322
Processo filho criado! PID: 330 || Pai: 322
Processo filho criado! PID: 331 || Pai: 322
Processo filho criado! PID: 332 || Pai: 322

root [ - ]* ps lf
  F S UID    PID  PPID  PGRP   SZ   RECV TTY   TIME CMD
  10 S  0    305   303   305    0   (wait) pm  p1  0:00 bash
  10 S  0    322   238   238    0   (wait) pm  p0  0:00 fork-ex
  10 S  0    323   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    324   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    325   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    326   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    327   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    328   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    329   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    330   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    331   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 S  0    332   322   238    0 (select) vfs  p0  0:00 fork-ex
  10 W  0    333   305   305    0           pm  p1  0:00 ps
  10 S  0    143    1   143    0   (tty) vfs  c0  0:00 getty
  10 S  0    144    1   144    0   (tty) vfs  c1  0:00 getty
  10 S  0    145    1   145    0   (tty) vfs  c2  0:00 getty
  10 S  0    146    1   146    0   (tty) vfs  c3  0:00 getty
  10 S  0    238   236   238    0   (wait) pm  p0  0:00 bash
  
```

Figura 6 – Verificando os processos filhos e o processo pai.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NFORKS 10 // Criamos 10 processos filhos

void do_nothing() {
    printf("Processo filho criado! PID: %d || Pai: %d\n", getpid(), getppid());
    sleep(20); // Mantém o processo ativo por 20 segundos para podermos visualizar
no `ps`
    exit(0);
}

int main(int argc, char *argv[]) {
    int pid[NFORKS], j, status, completed_processes = 0;
    pid_t terminated_pid;

    for (j = 0; j < NFORKS; j++) {
        pid[j] = fork();

        if (pid[j] < 0) {
            printf("Erro ao criar o processo filho.\n");
            exit(1);
        }

        if (pid[j] == 0) { // Processo filho
            do_nothing();
        }
    }

    // Esperando os processos filhos finalizarem
    while (completed_processes < NFORKS) {
        terminated_pid = wait(&status);
        completed_processes++;
    }
}
  
```

```

    printf("Processo filho finalizado (%d).\n", terminated_pid);
}

return 0;
}

```

Código 1 - Conteúdo do arquivo fork-ex-10pid.c.

Como podemos observar, o processo pai é o processo PID 322 e os filhos são os processos que vão de 323 até 332.

Para a próxima etapa, ainda com o código em execução, iremos usar o comando “kill 322” para encerrarmos o processo pai e vamos verificar novamente os processos com o comando “ps lf” para ver se os processos filhos se tornaram-se órfãos (Figura 7);

The image shows a terminal window with two panes. The left pane shows the output of a program that creates 10 child processes (PIDs 323-332) from a parent (PID 322). The right pane shows the command 'kill 322' being executed, followed by 'ps lf' which lists the remaining processes. In the 'ps lf' output, the PPID (Parent PID) for all listed processes is 1, indicating they are orphans. Handwritten red annotations include 'init' with an arrow pointing to PPID 1, and 'ÓRFÃOS' with a bracket pointing to the list of processes.

```

root [ ~/tarefa_5 ]* ./fork-ex-10pid
Processo filho criado! PID: 323 || Pai: 322
Processo filho criado! PID: 324 || Pai: 322
Processo filho criado! PID: 325 || Pai: 322
Processo filho criado! PID: 326 || Pai: 322
Processo filho criado! PID: 327 || Pai: 322
Processo filho criado! PID: 328 || Pai: 322
Processo filho criado! PID: 329 || Pai: 322
Processo filho criado! PID: 330 || Pai: 322
Processo filho criado! PID: 331 || Pai: 322
Processo filho criado! PID: 332 || Pai: 322
Terminated
root [ ~/tarefa_5 ]*

root [ - ]* kill 322
root [ - ]* ps lf
  F S UID    PID  PPID  PGRP   SZ   RECV ITTY  TIME CMD
  -- -- --
10 S 0      305    303    305     0   (wait) pm  p1  0:00 bash
10 S 0      323     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      324     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      325     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      326     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      327     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      328     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      329     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      330     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      331     1    238     0 (select) vfs  p0  0:00 fork-ex
10 S 0      332     1    238     0 (select) vfs  p0  0:00 fork-ex
10 W 0      334    305    305     0           pm  p1  0:00 ps
10 S 0      143     1    143     0   (tty) vfs  co  0:00 getty
10 S 0      144     1    144     0   (tty) vfs  c1  0:00 getty
10 S 0      145     1    145     0   (tty) vfs  c2  0:00 getty
10 S 0      146     1    146     0   (tty) vfs  c3  0:00 getty
10 S 0      238    236    238     0   (tty) vfs  p0  0:00 bash

```

Figura 7 – Encerramento do processo pai, e análise dos processos órfãos.

E o que de fato aconteceu após encerrarmos o processo pai?

Quando um processo pai é encerrado, seus processos filhos não desaparecem imediatamente. Em vez disso, eles se tornam processos órfãos e são adotados pelo processo init (ou seu equivalente no MINIX). Esses processos continuam em execução até que finalizam naturalmente ou sejam encerrados manualmente.

Na Figura 8, podemos observar que os processos filhos foram encerrados devido ao delay de 20 segundos definido no código. Esse tempo permitiu que analisássemos seu comportamento antes do término. Como os processos finalizaram automaticamente após esse período, isso caracteriza a técnica de "expiração". Já o encerramento manual do processo pai representa a técnica de "exterminar o processo".

```
root [ ~/tarefa_5 ]# ./fork-ex-10pid
Processo filho criado! PID: 323 || Pai: 322
Processo filho criado! PID: 324 || Pai: 322
Processo filho criado! PID: 325 || Pai: 322
Processo filho criado! PID: 326 || Pai: 322
Processo filho criado! PID: 327 || Pai: 322
Processo filho criado! PID: 328 || Pai: 322
Processo filho criado! PID: 329 || Pai: 322
Processo filho criado! PID: 330 || Pai: 322
Processo filho criado! PID: 331 || Pai: 322
Processo filho criado! PID: 332 || Pai: 322
Terminated
root [ ~/tarefa_5 ]#

root [ - ]# kill 322
root [ - ]# ps 1f
  F S UID PID PPID PGRP 82      RECV TTY  TIME CMD
  10 S 0 305 303 305 0 (wait) pw p1 0:00 bash
  10 S 0 323 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 324 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 325 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 326 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 327 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 328 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 329 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 330 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 331 1 238 0 (select) vfs p0 0:00 fork-ex
  10 S 0 332 1 238 0 (select) vfs p0 0:00 fork-ex
  10 W 0 334 305 305 0 pw p1 0:00 ps
  10 S 0 143 1 143 0 (tty) vfs co 0:00 getty
  10 S 0 144 1 144 0 (tty) vfs c1 0:00 getty
  10 S 0 145 1 145 0 (tty) vfs c2 0:00 getty
  10 S 0 146 1 146 0 (tty) vfs c3 0:00 getty
  10 S 0 239 230 238 0 (tty) vfs p0 0:00 bash

root [ - ]# ps 1f
  F S UID PID PPID PGRP 82      RECV TTY  TIME CMD
  10 S 0 305 303 305 0 (wait) pw p1 0:00 bash
  10 W 0 335 305 305 0 pw p1 0:00 ps
  10 S 0 143 1 143 0 (tty) vfs co 0:00 getty
  10 S 0 144 1 144 0 (tty) vfs c1 0:00 getty
  10 S 0 145 1 145 0 (tty) vfs c2 0:00 getty
  10 S 0 146 1 146 0 (tty) vfs c3 0:00 getty
  10 S 0 239 230 238 0 (tty) vfs p0 0:00 bash
```

Figura 8 – Verificando encerramento dos processos órfãos.

2. Referências

- [1] Dalton Matsuo Tavares, (2025) “PROJETO 4 – MINIX”, material disponibilizado no google classroom e SIGAA, Janeiro.